

Homework 3

by Petros Efraim Koukios, and Alex Ryström
Group 68

February 10, 2026

Problem 4: The Unisex Bathroom problem (40 points)

In this homework, concurrent programming using POSIX threads and semaphores was used to solve the Unisex Bathroom problem. The problem involves a bathroom, a shared resource, that may be used by either a number of men or women simultaneously, but never by both genders at the same time. The solution must also avoid deadlock and ensure fairness, meaning that no thread might be left waiting indefinitely. This program simulates people working and periodically using the bathroom.

Each thread runs in an infinite loop, alternating between working and using the bathroom. This makes the program behave more like a real system where access to the shared resource happens repeatedly over time. Random sleep intervals are used both before entering and while inside the bathroom, which increases the randomness.

Firstly, shared counters were used to keep track of how many men and women are currently inside the bathroom. These variables are accessed by multiple threads and must therefore be protected to avoid race conditions. Binary semaphores were used as mutexes to ensure that updates to these counters are performed atomically.

```
// shared counters
int men_inside_count = 0;
int women_inside_count = 0;

// mutex semaphores protecting the counters
sem_t men_count_lock;
sem_t women_count_lock;
```

Each gender has its own counter and lock, allowing multiple threads of the same gender to enter concurrently while still preserving correct counter updates.

The core requirement of the problem is that the bathroom must never be occupied by both genders at the same time. To achieve this we used a semaphore representing ownership of the bathroom. The first thread of a gender entering the bathroom locks this semaphore, and the last one leaving releases it.

```
// semaphore controlling bathroom ownership
sem_t gender_key;
```

For men, the logic is:

```
// man entering
sem_wait(&men_count_lock);
men_inside_count++;
if (men_inside_count == 1) {
    sem_wait(&gender_key);    // first man locks out women
}
sem_post(&men_count_lock);
```

```

// man leaving
sem_wait(&men_count_lock);
men_inside_count--;
if (men_inside_count == 0) {
    sem_post(&gender_key);    // last man releases bathroom
}
sem_post(&men_count_lock);

```

The same logic is mirrored for women using the `women_inside_count`. This mechanism ensures mutual exclusion between genders while still allowing multiple threads of the same gender to enter simultaneously.

To ensure fairness and prevent starvation, a semaphore called *gate* was introduced. Every thread must pass through this semaphore before attempting to enter the bathroom. This creates a queue-like ordering, preventing one gender from continuously overtaking the other.

```

// fairness gate
sem_t gate;

// usage before entering
sem_wait(&gate);
sem_wait(&men_count_lock);
men_inside_count++;
if (men_inside_count == 1)
    sem_wait(&gender_key);
sem_post(&men_count_lock);
sem_post(&gate);

```

This mechanism ensures that threads are served roughly in arrival order, preventing starvation of either gender.

In conclusion, the solution demonstrates how semaphores can be used for multiple synchronization roles: mutual exclusion (protecting shared counters), resource ownership control (bathroom availability), and fairness enforcement (turnstile). The program avoids deadlock by preventing circular waits, ensures that different genders never occupy the bathroom simultaneously, and guarantees that waiting threads eventually gain access. The simulation confirms correct concurrent behavior through printed trace messages.

```

PS C:\Users\petko\Documents\GitHub\Concurrent-Assignments\Assignment3> .\bathroom.exe
Woman 1 wants to use the bathroom.
>>> Woman 1 entered. (Women inside: 1)
Man 1 wants to use the bathroom.
<<< Woman 1 leaving. (Women inside: 0)
--- Bathroom is now EMPTY (Women unlocked it) ---
>>> Man 1 entered. (Men inside: 1)
Woman 5 wants to use the bathroom.
Woman 3 wants to use the bathroom.
Man 3 wants to use the bathroom.
Man 5 wants to use the bathroom.
<<< Man 1 leaving. (Men inside: 0)
--- Bathroom is now EMPTY (Men unlocked it) ---
>>> Woman 5 entered. (Women inside: 1)
>>> Woman 3 entered. (Women inside: 2)
Woman 2 wants to use the bathroom.
Man 2 wants to use the bathroom.
<<< Woman 3 leaving. (Women inside: 1)
Woman 4 wants to use the bathroom.
Man 4 wants to use the bathroom.
PS C:\Users\petko\Documents\GitHub\Concurrent-Assignments\Assignment3> |

```

Figure 1: Example execution output showing thread activity and synchronization behavior.